# Arculus Recovery v1.6.0

User Guide and Technical Reference

Offline BIP39/BIP32 recovery, encrypted seed backup, QR export, file-format reference, and operational security procedures.

| | |
|---|---|
| **Primary GUI** | Standalone HTML application |
| **Desktop GUI** | PySide6 WebEngine wrapper |
| **Desktop Package** | Tauri wrapper with native export bridge |
| **CLI** | Python derivation and export interface |
| **Generated** | June 7, 2026 |

# Arculus Recovery v1.6.0 User Guide

## Purpose

Arculus Recovery is an offline BIP39/BIP32 recovery and key-derivation tool. It helps users validate a mnemonic, derive addresses and private keys, import or export encrypted .arc seed backups, and produce local recovery exports.

The standalone HTML file is the canonical graphical app. The PySide6 GUI and Tauri desktop package render the same HTML interface. The Python CLI exposes the derivation engine for scripted or text-only sessions.

## Editions

```
Standalone HTML:
  Open Arculus_Recovery.html directly in a browser. It is self-contained and
  does not require Python, npm, Tauri, vendor files, a CDN, or network access.

Python GUI:
  Run "python Arculus_Recovery.py --gui" after installing requirements.txt.
  This opens the HTML interface in PySide6 WebEngine.

Python CLI:
  Run "python Arculus_Recovery.py" with --mnemonic and derivation flags.
  CLI output supports JSON, CSV, and TXT.

Tauri desktop package:
  Runs the same HTML app in a native WebView. Packaged exports use a native
  save bridge before falling back to browser-style download behavior.
```

## Before You Begin

Use the tool only on a trusted offline machine. Disconnect networking before entering or importing seed material. A network indicator is useful feedback, but physical disconnection is the authority.

Have these inputs ready:

- 12-word or 24-word mnemonic, or an encrypted .arc file.
- BIP39 passphrase if the wallet used one.
- Target coin.
- Derivation path and script type, if known.
- A known address or root fingerprint for verification.

A wrong passphrase, path, script type, or mnemonic can produce valid-looking but unrelated keys. Do not act on output until a known fingerprint or address matches.

## Quick Recovery Workflow

- Open the HTML app, Python GUI, or Tauri app on the offline machine.
- Enter the mnemonic, generate a test mnemonic, or import a .arc file.
- Click Validate Mnemonic and resolve any errors.
- Enter the BIP39 passphrase only if the wallet used one.
- Choose coin, script type, path, address count, and optional range.
- Click Derive Keys + Addresses.
- Verify the root fingerprint or a known address.
- Export only the material you need.
- Click Clear All, close the app, and power down the environment.

## Main Controls

```
Mnemonic entry:
  Use the word grid or paste a full mnemonic. The 12/24 selector controls active
  word slots. Generated, imported, and successfully derived manually entered
  seeds are masked and can be revealed only while Show Seed is held.

Passphrase:
  Optional BIP39 passphrase. This is not the .arc file password. It changes the
  derived wallet tree and produces no error when wrong.

Coin:
  Select Bitcoin, Litecoin, Dogecoin, Ethereum / ERC-20, or XRP.

Derivation path:
  Account-level BIP32 path. Arculus-native Bitcoin recovery defaults to m/0'.
  Standard wallet paths include m/44', m/49', m/84', and m/86' purpose paths.

Script type:
  Controls UTXO address encoding for Bitcoin, Litecoin, and Dogecoin. Auto
  infers the type from standard purpose values. Ethereum and XRP ignore UTXO
  script type.

Address count and range:
  Controls how many receiving and change addresses are derived. Use a range to
  inspect a specific index window.

Output tabs:
  Table View is for review and copy actions. Raw JSON contains the complete
  structured output. Extended Keys isolates root and account extended keys.

Export:
  Writes JSON, CSV, TXT, or PDF. These exports may contain private keys and are
  not encrypted by the .arc format.

Encrypted seed:
  Encrypt/Export Seed writes an authenticated .arc file. Import Seed decrypts a
  .arc file and loads the mnemonic into the hidden-seed workflow.

QR Export:
  Generates a black-on-white QR code for an address or supported payment URI.
  XRP addresses use a destination-tag workflow.

Settings:
  Select Light, Dark, Dark+, or Terminal theme. Theme preference is local app
  state only.
```

## Default Paths

```
  Coin       Default path       Notes
  ---------  -----------------  ------------------------------------------
  Bitcoin    m/0'               Arculus-native default
  Litecoin   m/84'/2'/0'        Native SegWit default
  Dogecoin   m/44'/3'/0'        Legacy BIP44 default
  Ethereum   m/44'/60'/0'       Also used by ERC-20 tokens
  XRP        m/44'/144'/0'      Destination tags are not derived
```

Common UTXO paths:

```
  Purpose   Script type      Bitcoin        Litecoin       Dogecoin
  -------   --------------   -------------  -------------  -------------
  BIP44     P2PKH            m/44'/0'/0'    m/44'/2'/0'    m/44'/3'/0'
  BIP49     P2WPKH-P2SH      m/49'/0'/0'    m/49'/2'/0'    m/49'/3'/0'
  BIP84     P2WPKH           m/84'/0'/0'    m/84'/2'/0'    m/84'/3'/0'
  BIP86     P2TR             m/86'/0'/0'    m/86'/2'/0'    disabled
```

## Export Sensitivity

JSON, CSV, TXT, and PDF exports can contain the mnemonic, private keys, WIF keys, and extended private keys. Treat them as funds-controlling secrets.

The .arc format encrypts only the mnemonic backup. It does not encrypt derived exports, PDF files, QR PNGs, or the BIP39 passphrase.

## End-of-Session Checklist

- The mnemonic checksum passed.

- Root fingerprint or known address was verified.
- Coin, path, script type, passphrase, count, and range were checked.
- Exports were stored on encrypted media.
- Clear All was clicked.
- The offline environment was powered down.

# Interface Guide

The screenshots below show the primary user-facing layout and theme variants. The HTML application is the canonical GUI surface. PySide6 and Tauri package the same interface for desktop use, so the screenshots apply to all GUI editions.

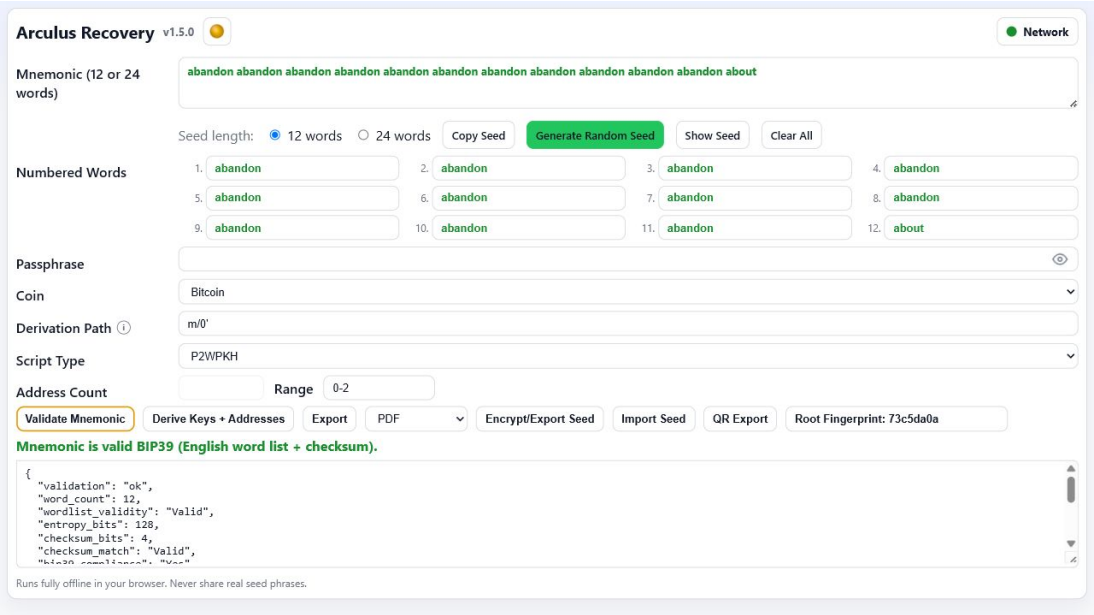


*Figure 1. Main recovery workspace with test-vector mnemonic validation controls.*

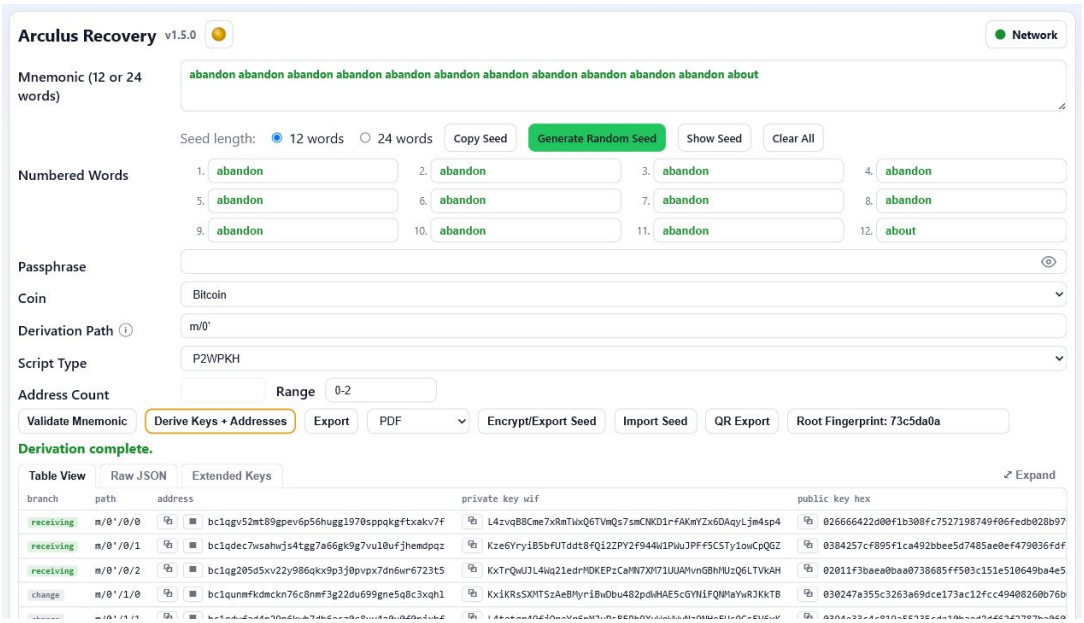


*Figure 2. Derived output table after address derivation, with output tabs and export controls.*

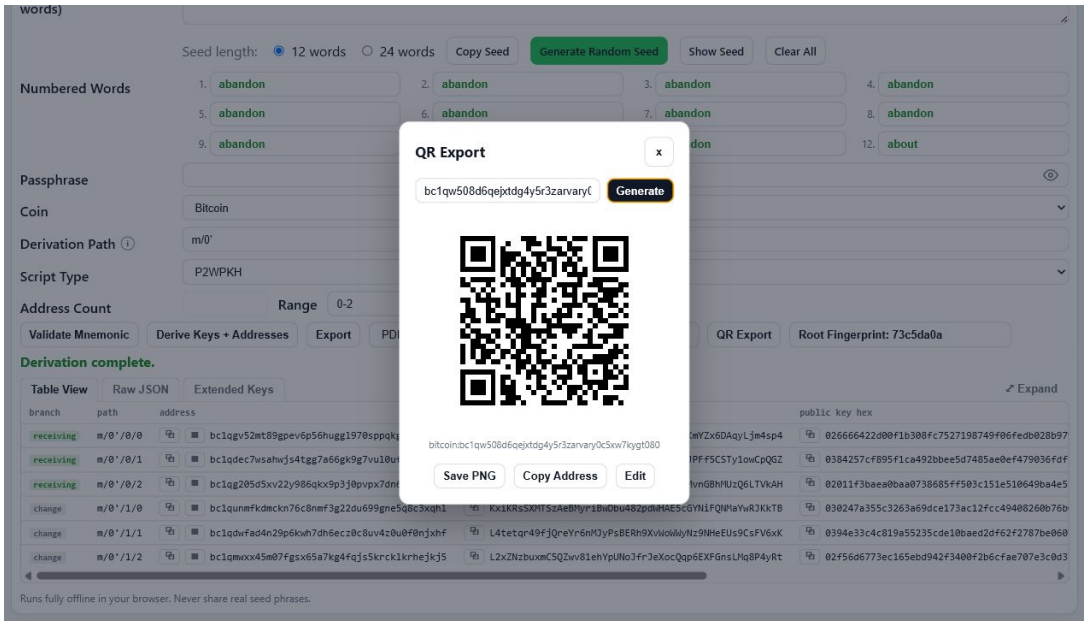


*Figure 3. QR Export modal generating an address QR code without external services.*

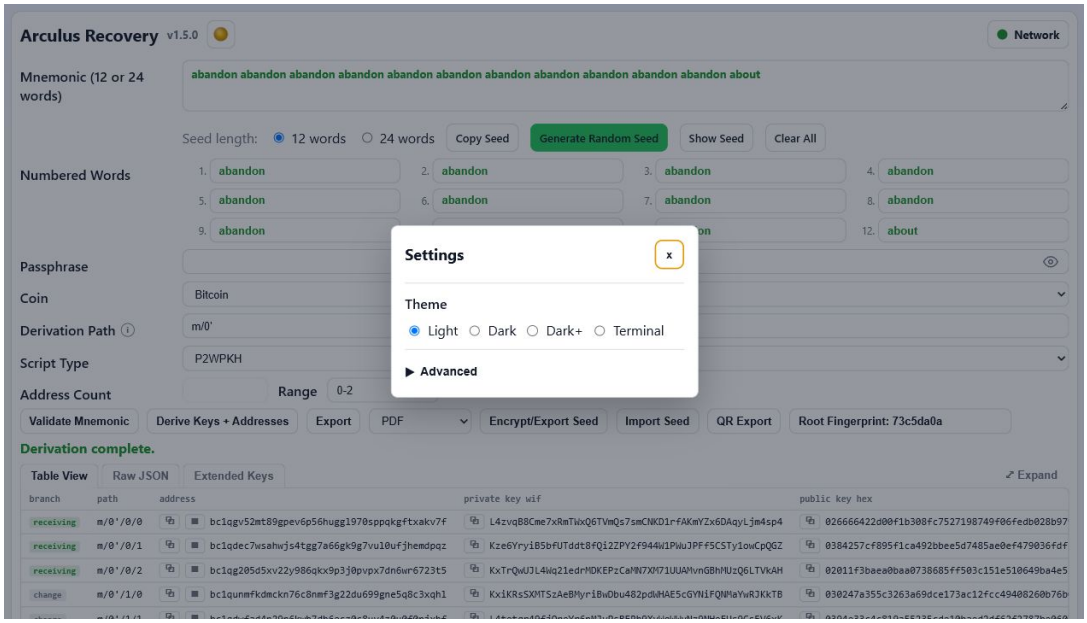


*Figure 4. Settings dialog with theme selection and auto-derive preference.*

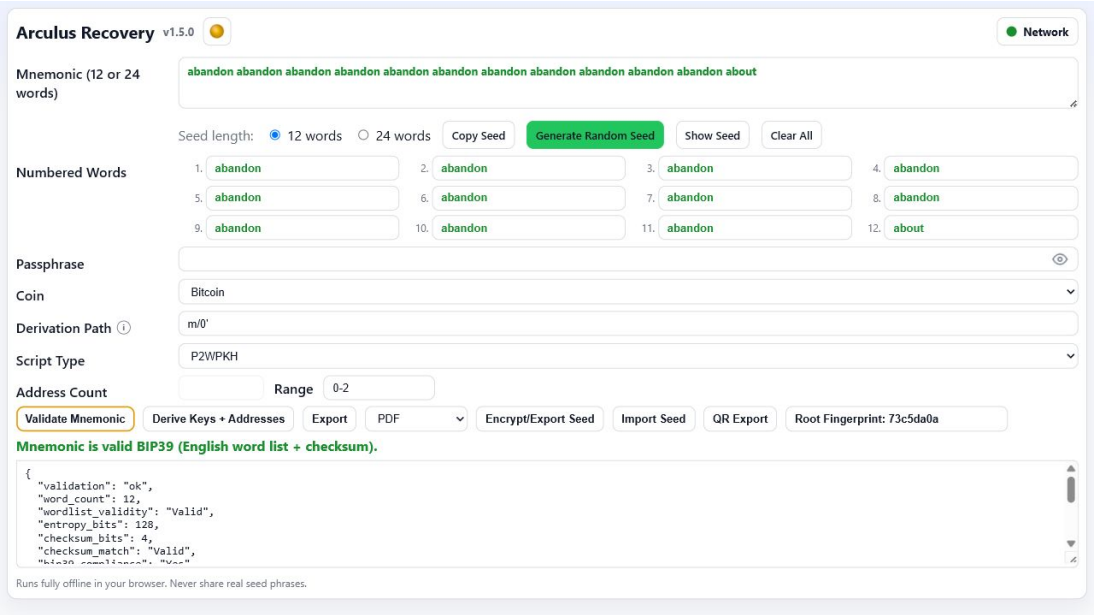


*Figure 5. Light theme interface.*

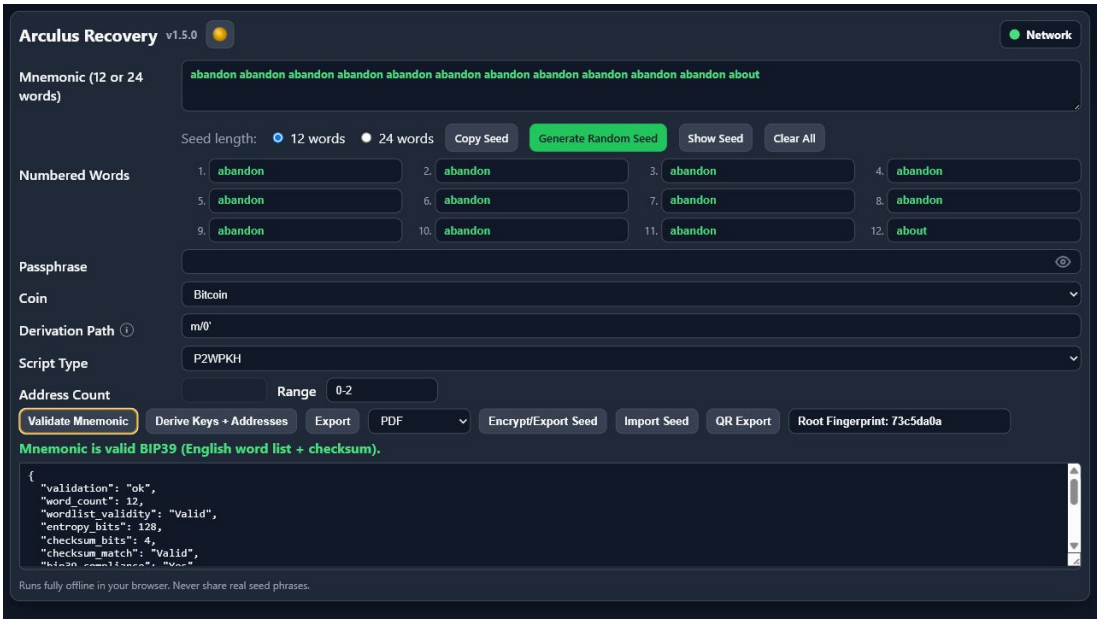


*Figure 6. Dark theme interface.*

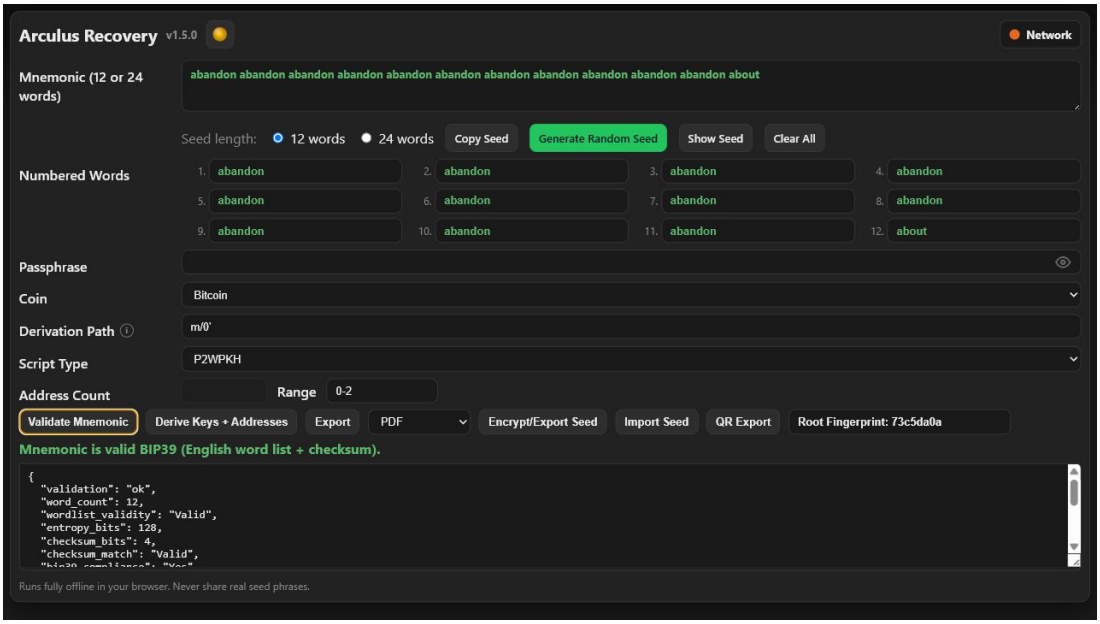


*Figure 7. Dark+ theme interface.*

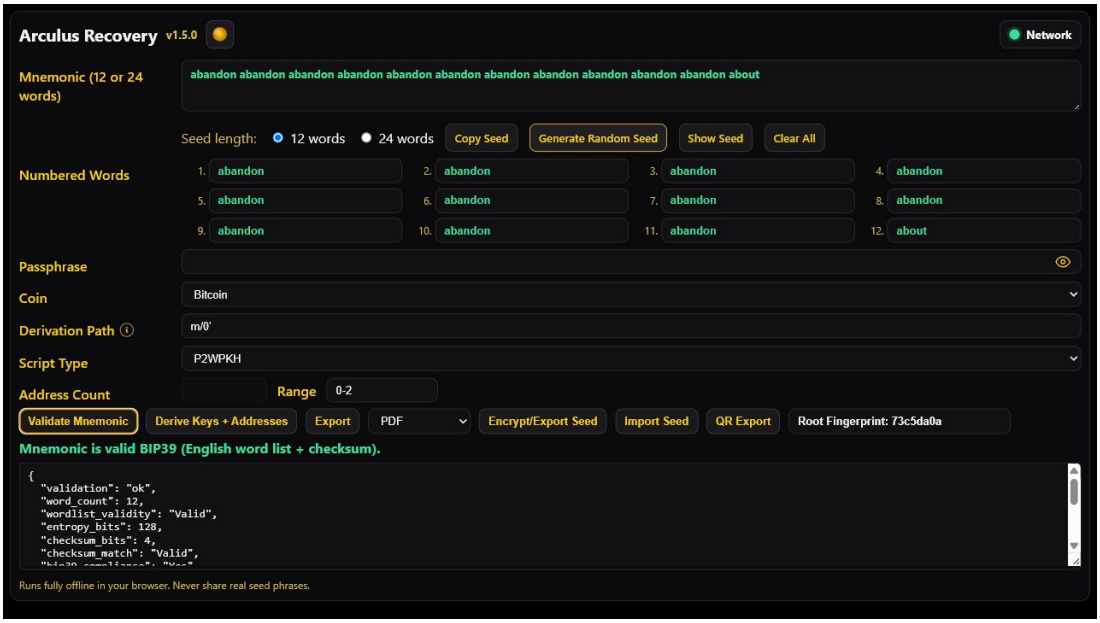


*Figure 8. Terminal theme interface.*

# Arculus Recovery v1.6.0 Recovery Procedures

## Scope

This document gives step-by-step recovery procedures for direct mnemonic entry, .arc import, CLI derivation, address discovery, and fund sweeping.

## Prerequisites

Before starting:

- The machine is offline.
- The operating system is trusted.
- The app hash was verified.
- You have the mnemonic or .arc file.
- You know whether a BIP39 passphrase was used, or have candidates.
- You know the target coin.
- You have a known address or root fingerprint if possible.

## Procedure 1: Direct Mnemonic Entry

- Open Arculus_Recovery.html, the PySide6 GUI, or the Tauri app.
- Select 12 or 24 words.
- Enter the mnemonic in the word grid or paste it into the input.
- Validate the mnemonic and fix any word-list or checksum errors.
- Enter the BIP39 passphrase only if the wallet used one.
- Select the coin.
- Select Auto script type unless you need a specific UTXO script.
- Enter the account path.
- Set the address count or range.
- Derive keys and addresses.
- Verify the root fingerprint or a known address.
- Export only the needed material.
- Clear All and close the app.

## Procedure 2: Recovery from .arc

- Transfer the .arc file to the offline machine.
- Open the app.
- Click Import Seed and select the .arc file.
- Enter the .arc password.
- If import succeeds, verify the root fingerprint.
- Enter the BIP39 passphrase separately if the wallet used one.
- Continue with coin, path, script type, and derivation.
- Clear All when finished.

The .arc password decrypts the file. It is not the BIP39 passphrase.

## Procedure 3: Python CLI Recovery

Example:

```
  python Arculus_Recovery.py \
    --mnemonic "abandon abandon abandon abandon abandon abandon abandon abandon abandon
abandon abandon about" \
    --passphrase "" \
    --derivation "m/84'/0'/0'" \
    --script-type p2wpkh \
    --coin bitcoin \
    --count 10 \
    --output-format json
```

When the originating wallet is unknown, use:

```
  python Arculus_Recovery.py \
    --mnemonic "word1 word2 ... word12" \
    --coin bitcoin \
    --all-common \
    --count 20 \
    --output-format txt
```

Redirect output only to trusted storage. CLI exports are plaintext.

## Standard Path Reference

| Coin | Script type | Path | Prefix |
|---|---|---|---|
| Bitcoin | P2PKH | m/44'/0'/0' | 1... |
| Bitcoin | P2WPKH-P2SH | m/49'/0'/0' | 3... |
| Bitcoin | P2WPKH | m/84'/0'/0' | bc1q... |
| Bitcoin | P2TR | m/86'/0'/0' | bc1p... |
| Bitcoin | Arculus default | m/0' | commonly bc1q... |
| Litecoin | P2PKH | m/44'/2'/0' | L... |
| Litecoin | P2WPKH-P2SH | m/49'/2'/0' | M... |
| Litecoin | P2WPKH | m/84'/2'/0' | ltc1q... |
| Litecoin | P2TR | m/86'/2'/0' | ltc1p... |
| Dogecoin | P2PKH | m/44'/3'/0' | D... |
| Dogecoin | P2WPKH-P2SH | m/49'/3'/0' | 9... or A... |
| Dogecoin | P2WPKH | m/84'/3'/0' | doge1q... |
| Ethereum | Account | m/44'/60'/0' | 0x... |
| XRP | Account | m/44'/144'/0' | r... |

## Address Discovery

Use this process when the path, script type, or passphrase is unknown:

1. Start with the empty BIP39 passphrase.
2. Try the most likely path for the coin.
3. Compare receiving addresses against a known address.
4. If no match, try known passphrase candidates.
5. Try standard paths for that coin.
6. Increase the address count to 25, 50, or 100.
7. Check the change branch.
8. Try account indices 1 and 2 if account 0 fails.
9. Record the confirmed path, script type, passphrase status, and root fingerprint.

## Sweeping Funds

Sweeping means importing derived private keys into a networked wallet and moving funds to a new wallet. This exposes the swept keys to the networked device.

1. Derive and export only keys for funded addresses.
2. Move the export to a trusted networked machine.
3. Import or sweep individual private keys, not whole extended private keys, where the wallet software supports it.
4. Send funds to a new wallet generated on trusted hardware.
5. Confirm transactions on-chain.
6. Treat the old mnemonic and exported keys as exposed after sweep.

## Verification Checklist

- Mnemonic checksum passed.

- Root fingerprint matches, or a known address matches.
- Passphrase value was checked.
- Path and script type match the originating wallet.
- Export destination is trusted.
- Clear All was used before ending the session.

# Arculus Recovery v1.6.0 Operational Security Guide

## Scope

This guide describes how to operate Arculus Recovery safely. The cryptographic engine is only useful if the surrounding computer, storage, display, and export workflow are controlled.

## Core Rules

- Use a trusted offline machine.
- Verify the tool before use.
- Disable network interfaces before seed entry.
- Avoid clipboard use for secrets.
- Export only what is required.
- Store exports on encrypted removable media.
- Clear the session and power down when finished.

## Airgap Requirements

Disconnect all network paths before entering seed material:

```
  - Remove Ethernet cables.
  - Disable Wi-Fi at the hardware or firmware level when possible.
  - Disable Bluetooth.
  - Disable cellular modems.
  - Do not reconnect while the mnemonic, passphrase, or private output is in
    memory or on screen.
```

The network indicator is a convenience signal. Physical disconnection remains the security control.

## Recommended Environment

```
Best option:
  A live operating system with no persistence, no hibernation, and no network.
  Run the app from a RAM-backed location and power off after the session.

Acceptable alternative:
  A dedicated offline machine that has never been used for general browsing or
  email. Persistent storage on this machine should be treated as sensitive after
  any recovery session.

Avoid:
  A daily-use networked computer, a browser profile with extensions enabled, a
  synced cloud desktop, or a machine that runs remote management software.
```

## Session Procedure

Pre-session:

- Prepare a private room and remove cameras from view of the screen.
- Boot the trusted offline environment.
- Verify the tool hash.
- Confirm all network interfaces are disconnected.
- Transfer any .arc file using removable media only.

During session:

- Enter or import the mnemonic.
- Validate the mnemonic before derivation.
- Enter the BIP39 passphrase only if the wallet used one.

- Verify the root fingerprint or a known address.
- Export only what is needed.

Post-session:

- Click Clear All.
- Close the app.
- Remove export media and store it securely.
- Power off the machine.

## Password Guidance for .arc Files

The .arc password protects against offline guessing when an attacker obtains the encrypted file. Use a high-entropy password:

```
Recommended:
  Six or more randomly selected Diceware-style words, or a randomly generated
  20-character password from a broad character set.

Avoid:
  PINs, dates, names, reused passwords, short phrases, keyboard patterns, or
  personal information.
```

Store the .arc password separately from the .arc file and separately from the mnemonic. If the password is lost, the .arc file cannot be decrypted.

## Output Handling

```
Category A, public or low sensitivity:
  Addresses and root fingerprints. These do not unlock funds, but addresses
  can reveal wallet activity when correlated with blockchain history.

Category B, funds-controlling:
  Mnemonics, BIP39 passphrases, .arc passwords, private keys, WIF keys,
  extended private keys, and any export containing those values.
```

Category B material must not be emailed, cloud-synced, photographed with a cloud-connected device, printed on a network printer, or stored on ordinary unencrypted storage.

## .arc Backup Verification

Every new .arc file should be verified before it is trusted:

- Export the .arc file.
- Import it again in the same offline session.
- Enter the password.
- Verify the root fingerprint.
- Store at least two copies on separate encrypted media.

Re-verify long-term backups periodically and re-export with the newest release when practical.

## Common Errors

```
Wrong BIP39 passphrase:
  Produces valid-looking but unrelated keys. Verify with a root fingerprint or
  known address.

Wrong derivation path:
  Produces valid addresses for the wrong wallet branch. Try standard paths and
  account indices systematically.

Leaving private exports on disk:
  Treat the old wallet as exposed if exports reached a networked or untrusted
  system.

Skipping .arc round-trip verification:
  Can leave you with a backup that cannot be decrypted later.
```

```
Storing .arc file and password together:
  Defeats the purpose of encryption.
```

# Arculus Recovery v1.6.0 BIP39 Passphrase Reference

## Scope

This document explains the BIP39 passphrase, its effect on recovery, and how it differs from the .arc encryption password.

## What It Is

The BIP39 passphrase is an optional string used during BIP39 seed derivation. It is sometimes called a "25th word", but it is not limited to BIP39 words and may be any Unicode string, including the empty string.

Derivation:

```
password_bytes = NFKD(mnemonic).encode("utf-8")
salt_bytes     = ("mnemonic" + NFKD(passphrase)).encode("utf-8")

seed = PBKDF2-HMAC-SHA512(
    password   = password_bytes,
    salt       = salt_bytes,
    iterations = 2048,
    dklen      = 64
)
```

Every distinct passphrase derives a different wallet tree from the same mnemonic. The empty passphrase is also a real passphrase value.

## What It Is Not

The BIP39 passphrase is not:

- The .arc encryption password.
- A login password for Arculus Recovery.
- Stored inside a .arc file.
- Checked by the BIP39 checksum.
- Recoverable from the mnemonic.

A wrong .arc password causes an authentication failure. A wrong BIP39 passphrase causes no error; it silently derives a different wallet.

## Whitespace, Case, and Unicode

Passphrases are case-sensitive and whitespace-sensitive.

```
"Password" != "password"
"secret"   != "secret "
```

BIP39 requires Unicode NFKD normalization before byte encoding. ASCII-only passphrases are the easiest to reproduce correctly across systems.

## Verification

The root fingerprint is the best practical confirmation that the mnemonic and passphrase combination is correct. Record the fingerprint when a wallet is set up or first verified.

At recovery time:

- Enter mnemonic and passphrase.
- Let the root fingerprint update.
- Compare it to the recorded value.
- Do not export or sweep funds if it does not match.

If no fingerprint is available, compare derived addresses against a known funded address from the wallet.

## Interface Behavior

The Passphrase field is masked by default. The visibility toggle reveals it only for visual checking. Clear All clears the field. The value is not saved as a preference.

The passphrase is not part of the hidden-seed workflow. When shown, it is visible on screen and readable to software that can inspect the page.

## Backup Requirements

Back up the passphrase separately from the mnemonic. A mnemonic without the passphrase recovers only the empty-passphrase wallet tree.

Recommended records:

- Mnemonic backup in one secure location.
- Passphrase backup in a different secure location.
- Root fingerprint stored with both records.
- Optional .arc file for encrypted mnemonic backup only.

No single location should hold both the mnemonic and passphrase unless the user intends that location to be sufficient for full recovery.

## Common Mistakes

```
Entering a passphrase when none was used:
  Produces the wrong wallet tree.

Omitting a passphrase that was used:
  Produces the empty-passphrase tree, not the funded wallet.

Changing case or whitespace:
  Produces a different seed with no warning.

Forgetting the passphrase:
  Cannot be repaired by Arculus Recovery. Recovery requires a backup or a
  feasible manual search of remembered candidates.
```

# Arculus Recovery v1.6.0 File Format Reference

## Scope

This reference describes files produced or consumed by Arculus Recovery:

- .arc encrypted seed backups
- JSON derived-output exports
- CSV derived-output exports
- TXT derived-output exports
- PDF exports
- QR PNG exports

All non-PDF structured text formats are UTF-8.

## .arc Encrypted Seed Files

A .arc file stores one BIP39 mnemonic in an encrypted, authenticated envelope. Current exports use armored V2:

```
Line 1: ARCULUS-ARC-V2
Line 2: base64(compact JSON bundle)
Line 3: optional trailing newline
```

Internal V2 bundle fields:

```
magic           "ARCULUS-ARC"
format          "arculus-encrypted-seed-v2"
version         2
created_at      UTC ISO 8601 timestamp
cipher          object with name and nonce_b64
kdf             object with name, hash, iterations, and salt_b64
ciphertext_b64  encrypted plaintext payload
mac_b64         HMAC-SHA512 authentication tag
```

Cipher object:

```
name            "HMAC-SHA512-CTR"
nonce_b64       base64 24-byte random nonce
```

KDF object:

```
name            "PBKDF2"
hash            "SHA-512"
iterations      1000000 for new exports
salt_b64        base64 32-byte random salt
```

Plaintext payload after decryption:

```
mnemonic        normalized space-separated BIP39 words
word_count      12 or 24
created_at      UTC ISO 8601 timestamp
```

Importers should ignore unknown plaintext fields.

## .arc Import Compatibility

The current app imports:

- ARCULUS-ARC-V2 armored files
- Raw JSON arculus-encrypted-seed-v2 files
- Legacy browser arculus-encrypted-seed-v1 files in the browser app
- Legacy arculus-encrypted-seed-python-v1 files
- Legacy headerless PBKDF2-SHA256/XOR-HMAC files

All new exports use ARCULUS-ARC-V2.

## Derived JSON Export

JSON is the most complete derived-output format. It contains top-level metadata, one or more account objects, and receiving/change address records.

Top-level fields:

```
coin
network
mnemonic
word_count
derivation
account_script_type_used
root_fingerprint
accounts
```

Account fields may include:

```
account_path
script_type
account_xprv / account_xpub
account_yprv / account_ypub
account_zprv / account_zpub
account_trprv / account_trpub
root_xprv / root_xpub
root_yprv / root_ypub
root_zprv / root_zpub
root_trprv / root_trpub
receiving
change
```

UTXO address records include:

```
path
branch
address
public_key_hex
private_key_hex
private_key_wif
```

Taproot records additionally include internal key, tweak, output key, output private key, output WIF, and output key parity fields.

Ethereum records include:

```
path
branch
address
ethereum_public_key_hex
private_key_hex
erc20_address
erc20_note
```

XRP records include:

```
path
branch
address
xrp_classic_address
public_key_hex
private_key_hex
xrp_note
```

The JSON export is plaintext and can contain the mnemonic and private keys.

## CSV Export

CSV is a flat row-per-address form of the same derived data. The first row is a header. Account-level fields are repeated on each row. Missing fields are empty cells. Values are quoted according to normal CSV rules when needed.

CSV exports are plaintext and may contain private keys.

## TXT Export

TXT is a human-readable labeled report. It includes top-level derivation metadata, account sections, extended keys, receiving addresses, and change addresses.

TXT exports are plaintext and may contain private keys.

## PDF Export

PDF export is generated by jsPDF in the HTML app. The standalone HTML file contains the PDF library inline, so no runtime network request is needed.

Typical PDF content:

- Title block
- Coin, app version, timestamp, and root fingerprint
- Extended keys section
- Address table
- Receiving/change branch labels
- Private key column appropriate for the coin

The default filename is:

```
Arculus_Key_Export_<Coin>.pdf
```

PDF exports are plaintext key documents when private fields are present.

## QR PNG Export

QR PNG export saves the rendered QR canvas as a PNG. Intended use is address or payment-URI export. A QR PNG is public if it contains only a receive address and destination tag. It becomes sensitive if the user manually encodes a private key, seed phrase, or other secret.

## Default Filenames

```
.arc   arculus_seed_<YYYYMMDD_HHMMSS>.arc
JSON   arculus_keys_<coin>.json
CSV    arculus_keys_<coin>.csv
TXT    arculus_keys_<coin>.txt
PDF    Arculus_Key_Export_<Coin>.pdf
PNG    qr_<sanitized_address>.png
```

Filenames are conventions only. Format detection should use content, not file extension.

# Arculus Recovery v1.6.0 Encryption Subsystem Internals

## Scope

This document specifies the current .arc encrypted seed format at the byte and algorithm level. It covers the V2 key schedule, stream cipher, MAC transcript, and import/export behavior.

## Design Goals

- No external cryptographic runtime dependency.
- Byte-compatible browser and Python implementations.
- Encrypt-then-MAC.
- MAC-bound metadata, including KDF parameters.
- Backward-compatible import for older .arc formats.

## Current Format Summary

```
Armor header:             ARCULUS-ARC-V2
Internal format ID:       arculus-encrypted-seed-v2
Version integer:          2
Password normalization:   Unicode NFKD, then UTF-8
KDF:                      PBKDF2-HMAC-SHA512
KDF output length:        64 bytes
New export iterations:    1000000
Minimum V2 import floor:  600000
Salt:                     32 random bytes
Nonce:                    24 random bytes
Cipher:                   HMAC-SHA512 counter stream
Authentication:           HMAC-SHA512
MAC length:               64 bytes
```

## Constants

```
ARC_MAGIC                 = "ARCULUS-ARC"
ARC_ARMOR_HEADER          = "ARCULUS-ARC-V2"
ARC_V2_FORMAT             = "arculus-encrypted-seed-v2"
ARC_V2_VERSION            = 2
ARC_V2_KDF_NAME           = "PBKDF2"
ARC_V2_KDF_HASH           = "SHA-512"
ARC_V2_KDF_ITERATIONS     = 1000000
ARC_V2_MIN_KDF_ITERATIONS = 600000
ARC_V2_SALT_BYTES         = 32
ARC_V2_NONCE_BYTES        = 24
ARC_V2_MAC_BYTES          = 64
ARC_V2_MASTER_KEY_BYTES   = 64
ARC_V2_CIPHER_NAME        = "HMAC-SHA512-CTR"
```

Domain labels:

```
ENC_LABEL    = b"Arculus ARC v2 encryption key"
MAC_LABEL    = b"Arculus ARC v2 authentication key"
STREAM_LABEL = b"Arculus ARC v2 stream\x00"
MAC_PREFIX   = b"Arculus ARC v2 MAC"
```

## Plaintext Payload

The encrypted plaintext is compact UTF-8 JSON:

```
{
  "mnemonic": "word1 word2 ... wordN",
  "word_count": 12,
  "created_at": "2026-06-13T00:00:00.000Z"
}
```

Only the mnemonic is protected by .arc encryption. The BIP39 passphrase and all derived-output exports are outside the .arc file.

## Key Schedule

- Normalize password:

```
password_bytes = NFKD(password).encode("utf-8")
```

- Derive master key:

```
master_key = PBKDF2-HMAC-SHA512(
    password=password_bytes,
    salt=salt,
    iterations=iterations,
    dklen=64
)
```

- Derive encryption key:

```
enc_key = HMAC-SHA512(master_key, ENC_LABEL)
```

- Derive authentication key:

```
mac_key = HMAC-SHA512(master_key, MAC_LABEL)
```

## Stream Cipher

For counter i = 0, 1, 2, ...

```
block_i = HMAC-SHA512(
    key=enc_key,
    msg=STREAM_LABEL || nonce || uint64_be(i)
)
```

The keystream is the concatenation of blocks truncated to the plaintext length. Encryption and decryption are XOR with the keystream.

## MAC Transcript

The MAC transcript is NUL-separated:

```
MAC_PREFIX
magic
format
version as decimal text
created_at
kdf.name
kdf.hash
kdf.iterations as decimal text
cipher.name
raw salt
raw nonce
raw ciphertext

transcript = b"\x00".join(parts)
mac = HMAC-SHA512(mac_key, transcript)
```

The MAC must be verified before decryption.

## Export Procedure

- Normalize and validate mnemonic.
- Generate 32-byte salt and 24-byte nonce with the platform CSPRNG.
- Create UTC timestamp.
- Derive enc_key and mac_key.
- Encrypt compact plaintext JSON.
- Build bundle metadata.
- Compute MAC over the transcript.
- Serialize compact JSON.
- Base64 encode the bundle and prepend ARCULUS-ARC-V2.

Two exports of the same mnemonic and password should produce different files because salt and nonce are freshly generated.

## Import Procedure

- Detect armor or legacy JSON.
- Parse bundle.
- Validate required fields.
- Reject V2 iteration counts below 600000.
- Decode salt, nonce, ciphertext, and MAC.
- Derive enc_key and mac_key from the supplied password.
- Recompute MAC and compare in constant time.
- Decrypt only after MAC success.
- Parse plaintext and validate the mnemonic.

MAC failure should not distinguish wrong password, corrupted file, or tampered metadata.

## Compatibility

| Format | Export | Import |
|---|---|---|
| ARCULUS-ARC-V2 armored V2 | yes | yes |
| Raw JSON arculus-encrypted-seed-v2 | no | yes |
| Legacy PBKDF2-SHA256/XOR-HMAC | no | yes |
| arculus-encrypted-seed-python-v1 | no | yes |
| arculus-encrypted-seed-v1 AES-GCM | no | browser implementation only |

Import a legacy file and export it again to migrate to current armored V2.

## Storage Limits

.arc protects the mnemonic at rest only. It does not protect:

- Mnemonic after decryption in memory.
- BIP39 passphrase.
- Private keys or extended private keys.
- Plaintext derived exports.
- Clipboard or screen contents.
- Files against an attacker who knows or guesses the password.

# Arculus Recovery v1.6.0 Derivation Engine Internals

## Scope

This document summarizes the deterministic derivation pipeline implemented by Arculus Recovery. The same conceptual pipeline is used by the standalone HTML app, PySide6 GUI, Tauri wrapper, and Python CLI.

## Pipeline

```
mnemonic input or CSPRNG entropy
    -> BIP39 word mapping and checksum validation
    -> Unicode NFKD normalization
    -> PBKDF2-HMAC-SHA512 BIP39 seed
    -> BIP32 master node
    -> account path derivation
    -> receiving and change branches
    -> per-index child keys
    -> address encoder
    -> structured output
```

The engine performs no balance lookup, chain scan, fee estimate, transaction signing, token lookup, or network I/O.

## BIP39 Generation

For 12 words:

```
entropy bits: 128
checksum bits: 4
```

For 24 words:

```
entropy bits: 256
checksum bits: 8
```

Entropy is generated by:

```
HTML:   crypto.getRandomValues
Python: os.urandom
```

The entropy is hashed with SHA256, checksum bits are appended, and the bitstream is split into 11-bit BIP39 word indices. Generated mnemonics are validated before being accepted.

## BIP39 Validation

Validation checks:

- Word count is 12 or 24.
- Every word is in the BIP39 English word list.
- Entropy and checksum bit lengths are correct.
- SHA256 checksum bits match.

A valid word with the wrong position can still produce a valid-looking mnemonic. Always verify against a root fingerprint or known address.

## BIP39 Seed

```
mnemonic_str   = NFKD(" ".join(words))
passphrase_str = NFKD(user_passphrase)

seed = PBKDF2-HMAC-SHA512(
    password=mnemonic_str.encode("utf-8"),
    salt=("mnemonic" + passphrase_str).encode("utf-8"),
    iterations=2048,
    dklen=64
)
```

The BIP39 passphrase is independent from the .arc encryption password.

## BIP32 Master Node

```
I  = HMAC-SHA512(key=b"Bitcoin seed", msg=seed)
IL = I[0:32]
IR = I[32:64]

master_private_key = parse256(IL)
master_chain_code  = IR
```

The master private scalar must be in the valid secp256k1 range.

Root fingerprint:

```
HASH160(compressed_master_public_key)[0:4]
```

## Path Syntax

Supported hardened markers:

```
m/84'/0'/0'
m/84h/0h/0h
m/84H/0H/0H
```

Output paths use apostrophe notation.

The configured path is treated as an account-level path. The engine derives:

```
receiving: <account path>/0/index
change:    <account path>/1/index
```

## Default Paths

```
bitcoin    m/0'
litecoin   m/84'/2'/0'
dogecoin   m/44'/3'/0'
ethereum   m/44'/60'/0'
xrp        m/44'/144'/0'
```

SLIP44 coin types:

```
bitcoin: 0
litecoin: 2
dogecoin: 3
ethereum: 60
xrp: 144
```

## Script Type Resolution

Explicit script types:

```
p2pkh
p2wpkh-p2sh
p2wpkh
p2tr
```

Auto mode maps purpose:

```
44' -> p2pkh
49' -> p2wpkh-p2sh
84' -> p2wpkh
86' -> p2tr
```

Ethereum and XRP use account-style outputs and ignore UTXO script type.

## Address Families

```
P2PKH:
  Base58Check(version || HASH160(compressed_pubkey))

P2WPKH-P2SH:
  Base58Check(p2sh_version || HASH160(OP_0 PUSH20 HASH160(pubkey)))

P2WPKH:
  Bech32 HRP with witness version 0 and 20-byte pubkey hash.

P2TR:
  BIP340/BIP341/BIP86 x-only internal key, TapTweak, and Bech32m witness
```

```
  version 1. Dogecoin Taproot is disabled.

Ethereum:
  Keccak-256 of the uncompressed secp256k1 public key, last 20 bytes, EIP-55
  checksum casing. ERC-20 tokens use the same address.

XRP:
  HASH160(compressed_pubkey), XRPL classic-address Base58 alphabet, r-prefix.
  Destination tags are not derived.
```

## Extended Keys

Extended private and public keys use BIP32 78-byte serialization followed by Base58Check. Script-specific variants such as ypub/zpub/trpub are emitted when applicable. Ethereum and XRP use standard xprv/xpub version bytes.

## CLI Reference

```
python Arculus_Recovery.py \
  --mnemonic "word1 word2 ... word12" \
  --passphrase "optional passphrase" \
  --derivation "m/84'/0'/0'" \
  --script-type p2wpkh \
  --coin bitcoin \
  --count 5 \
  --output-format txt
```

Flags:

```
--mnemonic        BIP39 mnemonic
--passphrase      Optional BIP39 passphrase
--derivation      Account-level BIP32 path
--all-common      Common purpose paths for the selected coin
--script-type     auto | p2pkh | p2wpkh-p2sh | p2wpkh | p2tr
--count           Address count per branch
--coin            bitcoin | litecoin | dogecoin | ethereum | xrp
--testnet         Supported testnet mode
--output-format   json | csv | txt
--gui             Launch GUI
--version         Print Python core version
```

## Failure Modes

Hard errors include:

- Unsupported word count.
- Unknown BIP39 word.
- Checksum mismatch.
- Invalid path component.
- Unsupported coin/script combination.
- Unsupported testnet configuration.
- Invalid secp256k1 scalar.
- Taproot output invalidity edge cases.

Wrong but valid inputs, such as a wrong passphrase or wrong path, do not produce errors. They produce a different wallet tree.

# Arculus Recovery v1.6.0 QR Export Reference

## Scope

The QR Export subsystem generates QR codes locally from addresses or supported payment URI strings. It is available in the HTML app and GUI surfaces that render the HTML app. It is not part of the Python CLI.

## Design

The QR encoder is embedded in the HTML application. It does not use an external library, CDN, server, or network request.

The intended payload is a receive address, optionally converted to a payment URI for wallet compatibility. Users can type arbitrary text, so they must not encode private keys, seed phrases, .arc contents, or other secrets.

## Rendering

Default rendering:

```
foreground: black
background: white
quiet zone: 4 modules
normal canvas: 256 px
XRP canvas: 320 px
```

QR codes remain black-on-white regardless of the selected theme because many wallet scanners fail on low-contrast or colored QR codes.

## URI Detection

If the input already begins with a URI scheme, it is passed through unchanged. Otherwise, the app detects common address formats:

```
Bitcoin:
  1..., 3..., bc1q..., bc1p...
  Encoded as bitcoin:<address>

Litecoin:
  L..., M..., ltc1q..., ltc1p...
  Encoded as litecoin:<address>

Dogecoin:
  D..., A..., 9...
  Encoded as dogecoin:<address>

Ethereum / ERC-20:
  0x followed by 40 hex characters
  Encoded as ethereum:<address>

XRP:
  r... classic address
  Uses the XRP destination-tag workflow.
```

Unknown strings are encoded as entered.

## XRP Destination Tags

Many exchanges use one shared XRP deposit address plus a destination tag. If a tag is required and omitted, funds may arrive at the exchange without being credited to the user.

When an XRP classic address is detected:

- The app prompts for a destination tag.
- If a tag is provided, the QR payload includes the address and dt query.
- If no tag is needed, the app renders a tagless-compatible XRP payload.
- The displayed label shows what was encoded.

The tool cannot know whether a given XRP address requires a tag. The exchange or custodian must provide that information.

## Entry Points

```
QR Export button:
  Opens a blank QR modal for manual address entry.

Table View row button:
  Opens the QR modal prefilled with the row address.
```

## Actions

```
Save PNG:
  Saves the rendered QR image. In Tauri builds, this uses the native export
  bridge when available.

Copy Address:
  Copies the original address text, not necessarily the URI payload.

Edit:
  Returns to the address or XRP tag input without closing the modal.
```

## Capacity and Failure

The encoder supports the address and URI lengths produced by this tool. If an input exceeds the supported QR capacity, the modal shows an error and does not render a partial code.

## Security Notes

Address QR codes are generally public information. QR codes that encode private keys, seed phrases, or other secrets are private-key artifacts and must be handled under the same controls as plaintext exports.

Do not scan a QR code with a networked device while seed material or private keys are still displayed on the offline machine.

# Arculus Recovery v1.6.0 Canonical Test Vector Suite

## Scope

This file records compact known-good vectors for validating the derivation and .arc encryption subsystems. The full implementation should continue to be tested against the repository's Python and HTML code paths.

## Conventions

All hex strings are lowercase. Paths use apostrophe notation for hardened components. Addresses use their standard encodings: Base58Check, Bech32, Bech32m, EIP-55, or XRPL classic Base58.

## BIP39 Mnemonics

```
TV1:
  mnemonic: abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon
abandon about
  passphrase: empty
  word_count: 12
  seed:
    5eb00bbddcf069084889a8ab9155568165f5c453ccb85e70811aaed6f6da5fc1
    9a5ac40b389cd370d086206dec8aa6c43daea6690f20ad3d8d48b2d2ce9e38e4
  root_fingerprint: 73c5da0a

TV1-T:
  mnemonic: same as TV1
  passphrase: TREZOR
  seed:
    c55257c360c07c72029aebc1b53c05ed0362ada38ead3e3e9efa3708e5349553
    1f09a6987599d18264c1e1c92f2cf141630c7a3c4ab7c81b2f001698e7463b04
  root_fingerprint: b4e3f5ed

TV2:
  mnemonic: abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon
abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon
abandon abandon art
  passphrase: empty
  word_count: 24
  seed:
    408b285c123836004f4b8842c89324c1f01382450c0d439af345ba7fc49acf70
    5489c6fc77dbd4e3dc1dd8cc6bc9f043db8ada1e243c4a0eafb290d399480840
  root_fingerprint: 5436d724
```

## Bitcoin Vectors, TV1

```
P2PKH, m/44'/0'/0'/0/0:
  address: 1LqBGSKuX5yYUonjxT5qGfpUsXKYYWeabA
  public_key_hex: 03aaeb52dd7494c361049de67cc680e83ebcbbbdbeb13637d92cd845f70308af5e
  private_key_hex: e284129cc0922579a535bbf4d1a3b25773090d28c909bc0fed73b5e0222cc372
  private_key_wif: L4p2b9VAf8k5aUahF1JCJUzZkgNEAqLfq8DDdQiyAprQAKSbu8hf

P2WPKH-P2SH, m/49'/0'/0'/0/0:
  address: 37VucYSaXLCAsxYyAPfbSi9eh4iEcbShgf
  public_key_hex: 039b3b694b8fc5b5e07fb069c783cac754f5d38c3e08bed1960e31fdb1dda35c24
  private_key_hex: 508c73a06f6b6c817238ba61be232f5080ea4616c54f94771156934666d38ee3
  private_key_wif: KyvHbRLNXfXaHuZb3QRaeqA5wovkjg4RuUpFGCxdH5UWc1Foih9o

P2WPKH, m/84'/0'/0'/0/0:
  address: bc1qcr8te4kr609gcawutmrza0j4xv80jy8z306fyu
  public_key_hex: 0330d54fd0dd420a6e5f8d3624f5f3482cae350f79d5f0753bf5beef9c2d91af3c
  private_key_hex: 4604b4b710fe91f584fff084e1a9159fe4f8408fff380596a604948474ce4fa3
  private_key_wif: KyZpNDKnfs94vbrwhJneDi77V6jF64PWPF8x5cdJb8ifgg2DUc9d

P2TR, m/86'/0'/0'/0/0:
  address: bc1p5cyxnuxmeuwuvkwfem96lqzszd02n6xdcjrs20cac6yqjjwudpxqkedrcr
  public_key_hex: 03cc8a4bc64d897bddc5fbc2f670f7a8ba0b386779106cf1223c6fc5d7cd6fc115
  private_key_hex: 41f41d69260df4cf277826a9b65a3717e4eeddbeedf637f212ca096576479361
  private_key_wif: KyRv5iFPHG7iB5E4CqvMzH3WFJVhbfYK4VY7XAedd9Ys69mEsPLQ
```

## Other Coin Vectors, TV1

```
Litecoin P2WPKH, m/84'/2'/0'/0/0:
  address: ltc1qjmxnz78nmc8nq77wuxh25n2es7rzm5c2rkk4wh
  public_key_hex: 02e49c9b9b5d0f127235dc26a0c252814c52fb333d651a946773f59d72c2da9904
  private_key_hex: 4ab54480cbbaa53d5d3ba43a3e85d221df085490e88346ad11579e17000db7bb
  private_key_wif: T5ZCYhLqXu6EJKk2nhjvwsaLH357CisixhLGWpKXEiqWTUtzte6o

Dogecoin P2PKH, m/44'/3'/0'/0/0:
  address: DBus3bamQjgJULBJtYXpEzDWQRwF5iwxgC
  public_key_hex: 02cc6b0dc33aabcf3a23643e5e2919a80c50fb3dd2129ce409bbc5f0d4643d05e0
  private_key_hex: 21f5e16d57b9b70a1625020b59a85fa9342de9c103af3dd9f7b94393a4ac2f46
  private_key_wif: QPkeC1ZfHx3c9g7WTj9cQ8gnvk2iSAfAcbq1aVAWjNTwDAKfZUzx

Ethereum, m/44'/60'/0'/0/0:
  address: 0x9858EfFD232B4033E47d90003D41EC34EcaEda94
  public_key_hex: 0237b0bb7a8288d38ed49a524b5dc98cff3eb5ca824c9f9dc0dfdb3d9cd600f299
  private_key_hex: 1ab42cc412b618bdea3a599e3c9bae199ebf030895b039e9db1e30dafb12b727

XRP, m/44'/144'/0'/0/0:
  address: rHsMGQEkVNJmpGWs8XUBoTBiAAbwxZN5v3
  public_key_hex: 031d68bc1a142e6766b2bdfb006ccfe135ef2e0e2e94abb5cf5c9ab6104776fbae
  private_key_hex: 90802a50aa84efb6cdb225f17c27616ea94048c179142fecf03f4712a07ea7a4
```

## .arc Encryption Checks

New exports must:

```
  - Start with ARCULUS-ARC-V2.
  - Use magic ARCULUS-ARC.
  - Use format arculus-encrypted-seed-v2.
  - Use version 2.
  - Use PBKDF2 / SHA-512.
  - Use 1000000 KDF iterations.
  - Use a 32-byte salt.
  - Use a 24-byte nonce.
  - Use a 64-byte MAC.
  - Produce different ciphertext for repeated exports of the same mnemonic and
    password, except with negligible probability.
```

Round-trip tests:

```
  - Correct password decrypts to the original normalized mnemonic.
  - Wrong password fails authentication.
  - Any bit flip in ciphertext, MAC, salt, nonce, created_at, or iteration
    count fails authentication.
  - V2 iteration counts below 600000 are rejected.
```

## Negative Tests

The implementation must reject:

- 1, 3, 11, 13, or other unsupported word counts.
- Unknown BIP39 words.
- Valid words with invalid checksum.
- Invalid path syntax such as m//0'.
- Non-numeric path components.
- 32-bit index overflow.
- Dogecoin Taproot.
- Unsupported testnet configurations.

## Compliance Criterion

An implementation is compatible only if it matches addresses, private keys, public keys, root fingerprints, extended keys, .arc structure, and rejection behavior for the relevant vectors. Matching addresses alone is not sufficient.